Functions

In Python, a function is a block of **reusable code** that performs a **specific task.** Functions also allow programmers to

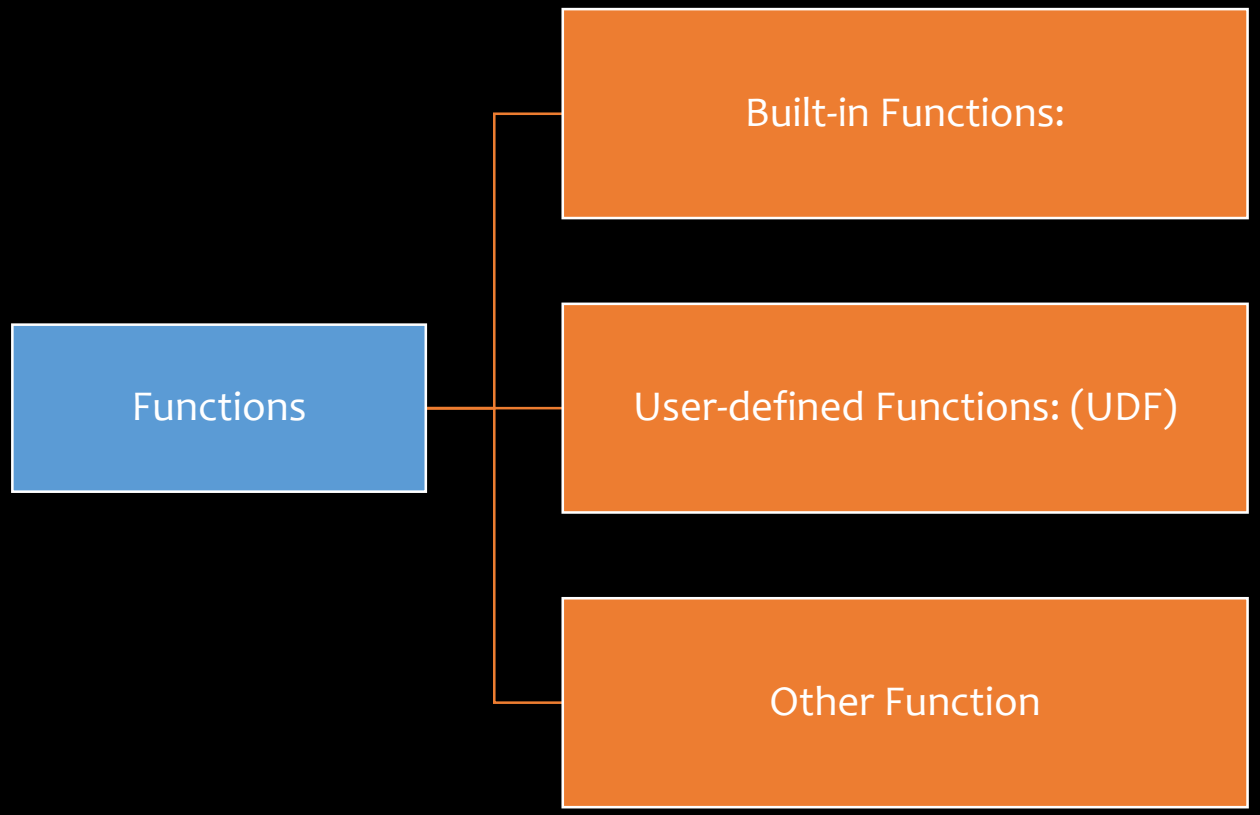
- break down your program into smaller, modular pieces,
- making your code more organized,
- easier to **understand** and easier to **maintain**.
- And it starts with **def** keyword

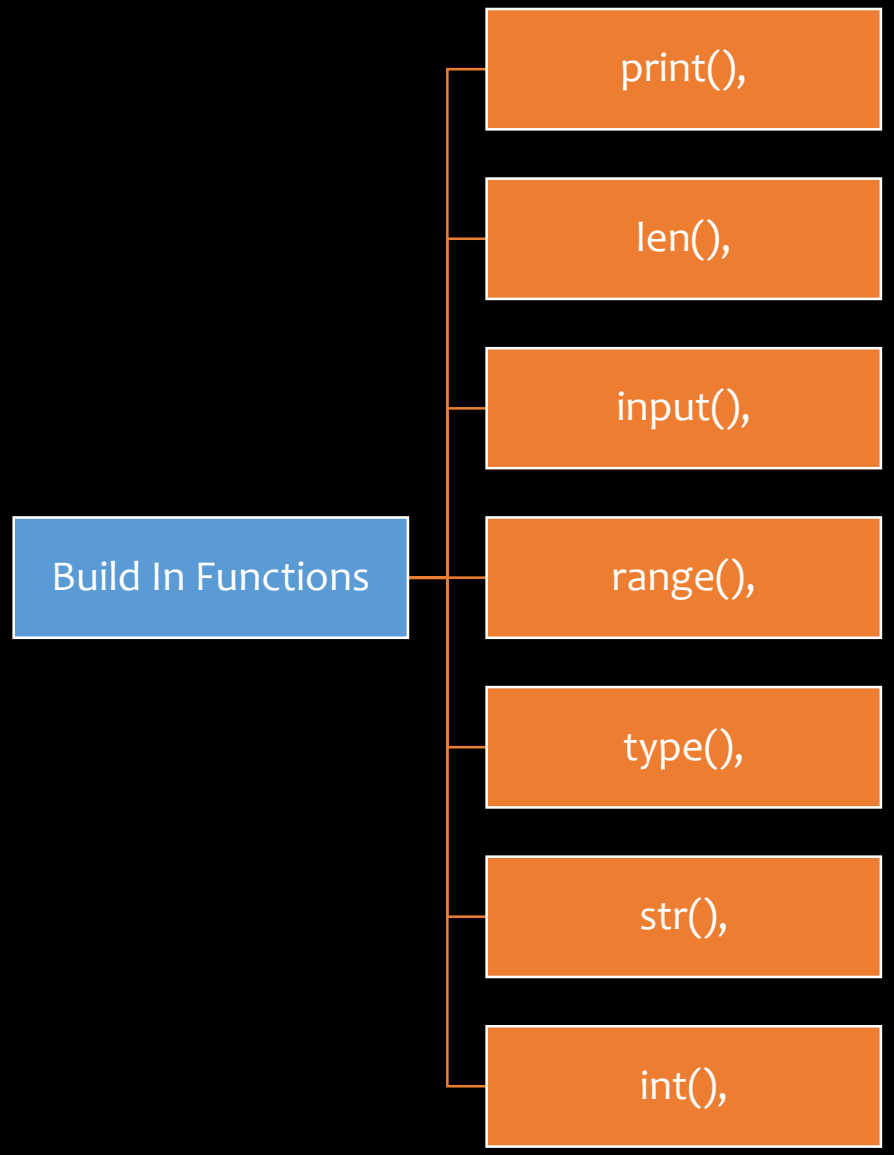
Python functions may contain attributes or logical structure or both.

```
2  def my_function(fname, lname):
3      print(fname + " " + lname)
4
5  my_function("Sunil", "Chavan")
```

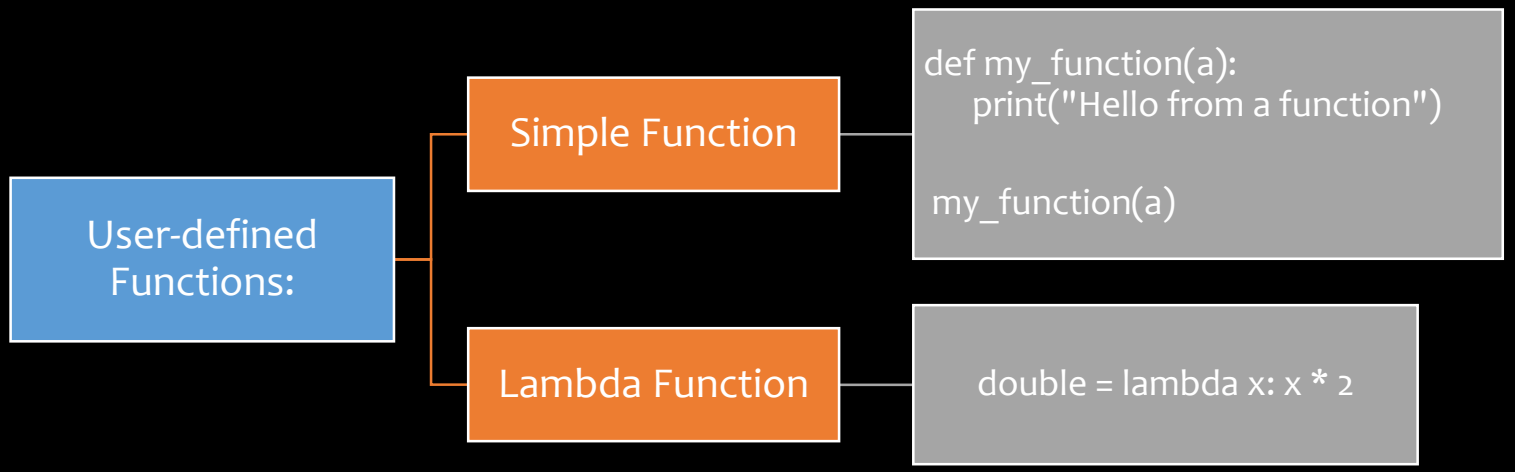
```
Sunil Chavan
```

Different Type of Functions



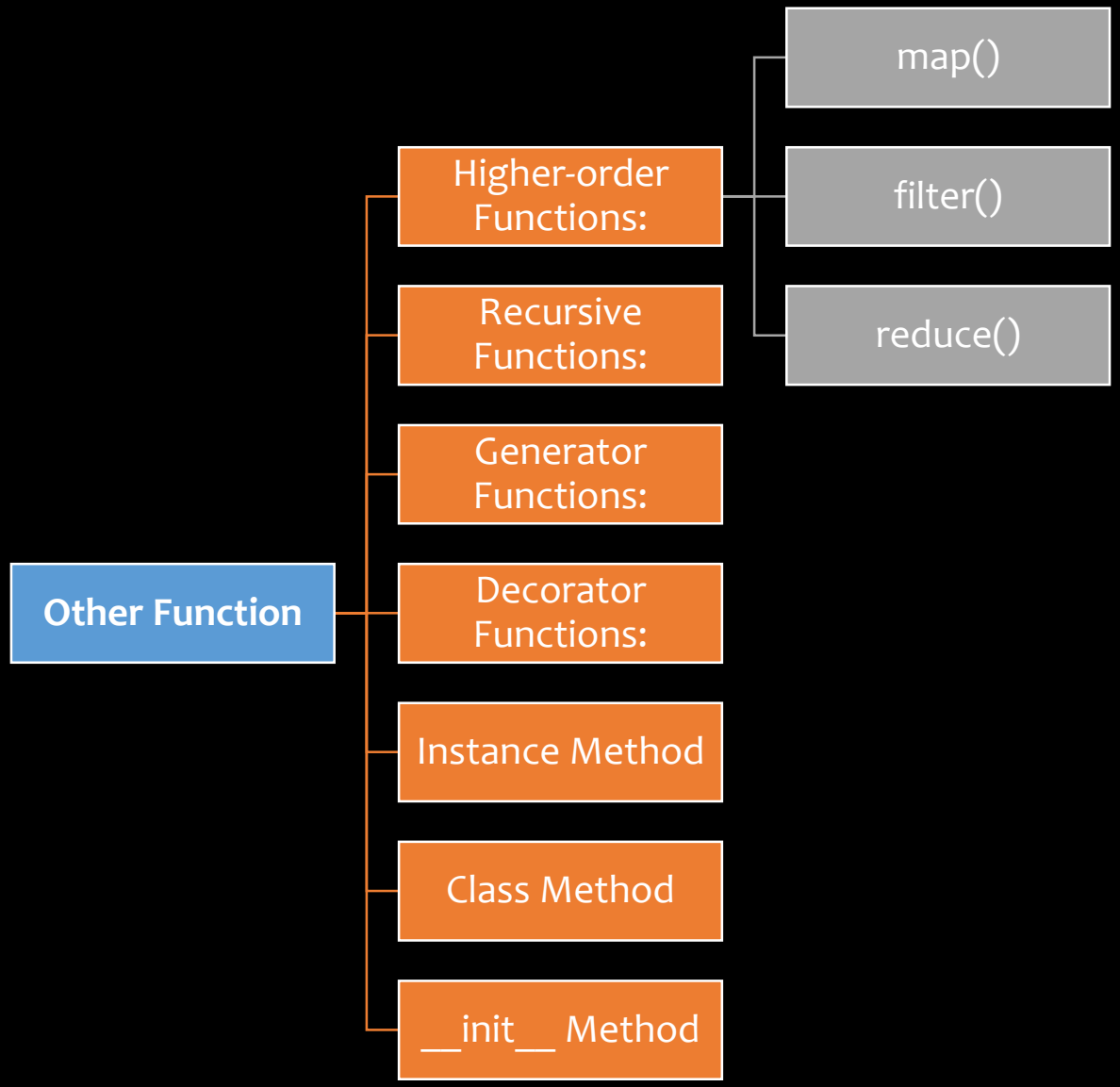


These are in build functions that are always available in Python, no matter which modules are imported.



User Defined function(UDF)

is a block of code that only runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result. In Python a function is defined using the **def** or **lambda** keyword :


Other Function
Higher-order Functions:
map()
filter()
reduce()
Recursive Functions:
Generator Functions:
Decorator Functions:
Instance Method
Class Method
__init__ Method

Higher-Order Functions

Think of a **higher-order function** as a function that has superpowers: instead of just working with normal data (like numbers or words), it can **work with other functions**.

- **reduce()**
- **map()**
- **filter()**

The **reduce(fun,seq)**

In simple terms, **reduce()** takes a whole list of items and **squashes them down into a single final answer**.

It is not a built-in function in Python; it exists in the **functools** module (so you must import it using from functools import reduce)

reduce(function, iterable)

```
'''Reduce'''
from functools import reduce

numbers = [1, 2, 3, 4, 5]

def multiply(x, y):
    return x * y

reduce(multiply,numbers) # Nothing
```

The **map()**

This function takes a function and iterables as input and applies the function to each item of the iterable(s). It returns an iterator that yields the results.

Here's the syntax: map(function, iterable1)

```
'MAP'
# Example 2: Squaring each number in a list using map

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
def square_number(x):
    return x ** 2


Var_01 = map(square_number,numbers)
list(Var_01)   # iterable

print(map(square_number,numbers))   # iterator Object
```

The **filter()** This function takes a function and an iterable as input(argument) and returns an iterator yielding those items of the iterable for which the function returns true.

Here's the syntax: filter(function, iterable)

```
# filter(function, iterable)'''
# find even number by using filer'

numbersList = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
def newFunc(i):
    if i%2!=0:
        return i

Vaue_01 = filter(newFunc,numbersList)
Vaue_01 = list(Vaue_01)
Print(filter(newFunc,numbersList)) # Itterable Object
```

Instance Variables (self.name, self.breed):

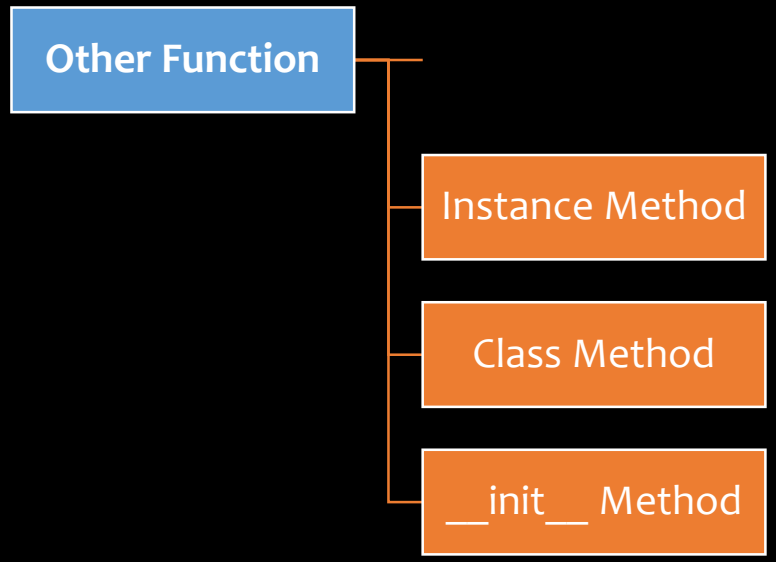
Defined inside __init__ using self.
They belong to that *specific* object. dog1 has its own name, and dog2 has its own name.

```
class Dog:
    def __init__(self, name, breed):
        # 1. Instance Variables (unique to each individual dog
object)
        self.name = name
        self.breed = breed
```

```
# --- How it works ---

# Creating two different dog objects (instances)
dog1 = Dog("Buddy", "Golden Retriever")
dog2 = Dog("Max", "German Shepherd")

# Accessing instance variables
print(dog1.name)   # Output: Buddy
print(dog2.breed)  # Output: Golden Retriever
```


Other Function
Instance Method
Class Method
__init__ Method

```python
class Dog:
    def __init__(self, name, breed):
        # 1. Instance Variables (unique to each individual dog object)
        self.name = name
        self.breed = breed

    # 2. Instance Method (uses the object's specific data)
    def bark(self):
        return f"{self.name} says Woof!“

  def Colour(self, colour):
    # Fixed f-string: changed self.name to {self.name}
    return f"{self.name} has {colour} color."
```

Instance Method:
Based on the provided image, an **Instance Method** is a regular function written inside a class that takes self as its first parameter. Because of this, it can access and use the instance variables belonging to that class.

```python
# --- How it works ---

# Creating two different dog objects (instances)
dog1 = Dog("Buddy", "Golden Retriever")
dog2 = Dog("Max", "German Shepherd")

# Calling the instance method
print(dog1.bark()) # Output: Buddy says Woof!
print(dog2.bark()) # Output: Max says Woof!

# 1. Create the object
Dog_03 = Dog("Moti", "Labra")

# 2. Call the instance method correctly (don't pass 'self', pass a real string)
print(Dog_03. Colour("Brown"))
```

```python
class Dog:
    def __init__(self, name, breed):
        # 1. Instance Variables (unique to each individual dog object)
        self.name = name
        self.breed = breed

    # 2. Instance Method (uses the object's specific data)
    def bark(self):
        return f"{self.name} says Woof!"
```

```python
# --- How it works ---

# Creating two different dog objects (instances)
dog1 = Dog("Buddy", "Golden Retriever")
dog2 = Dog("Max", "German Shepherd")

# Accessing instance variables
print(dog1.name)   # Output: Buddy
print(dog2.breed)  # Output: Golden Retriever

# Calling the instance method
print(dog1.bark()) # Output: Buddy says Woof!
print(dog2.bark()) # Output: Max says Woof!
```

Class Variable

```python
class Student:
    school_name = "Greenwood High"  # Class variable
```

Student Defined directly inside the class body. It is shared across all objects created from that class. If you change it, it affects all instances.

```python
s1 = Student() # object of
s2 = Student() # object of

# Both point to the shared class variable
print(s1.school_name)  # Output: Greenwood High
print(s2.school_name)  # Output: Greenwood High

# Change the class variable directly through the class
Student.school_name = "Sunrise Academy"

# Notice how it automatically updates for ALL existing instances!
print(s1.school_name)  # Output: Sunrise Academy
print(s2.school_name)  # Output: Sunrise Academy
```

```python
class BankAccount:
    # Class Variable: Shared interest rate for all accounts
    interest_rate = 0.04  # 4%

    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    # Class Method: Automatically increases the rate
    @classmethod
    def increase_interest_rate(cls):
        cls.interest_rate = cls.interest_rate + (cls.interest_rate * 0.02)
```

```python
# --- How to use it ---

# 1. Check the current bank interest rate
print(BankAccount.interest_rate)  # Output: 0.04

# 2. Call it WITHOUT arguments because it calculates it automatically
BankAccount.increase_interest_rate()

# 3. See the updated rate (0.04 plus 2% of 0.04)
print(BankAccount.interest_rate)  # Output: 0.0408
```

Lambda Functions:

Syntax of lambda function

```
func_One = lambda arguments : expression
func_One = lambda a         : a + 10

func_Two = lambda arguments : true_value if condition else false_value
func_Two = lambda x,y       :     x      if    x>y    else      y
```

END OF FUNCTIONS

Parameters or Arguments?

A parameter or argument is the variable listed inside the parentheses in the function definition. Or it is the value that is sent to the function when it is called.

```
2  def my_function(fname, lname):
3      print(fname + " " + lname)
4
5  my_function("Sunil", "Chavan")
```

```
Sunil Chavan
```

Arguments

- Positional Argument
- Arbitrary Positional Arguments, *args
- Keyword Arguments
- Arbitrary Keyword Arguments, **kwargs

```
2  def my_function(fname, lname):
3      print(fname + " " + lname)
4
5  my_function("Sunil", "Chavan")
```

```
Sunil Chavan
```

```
2  def my_function(*kids):
3      print("The youngest child is " + kids[1])
4  my_function("Amol", "Rajesh", "Sunil")
```

```
The youngest child is Rajesh
```

```
3  def my_function(child3, child2, child1):
4      print("The youngest child is " + child3)
5
6  my_function(child1 = "Raejsh", child2 = "SUnil", child3 = "Savita")
7
```

```
The youngest child is Savita
```

```
3  def my_function(**kid):
4      print("The youngest child is " + kid["child2"])
5
6  my_function(child1 = "Raejsh", child2 = "SUnil", child3 = "Savita")
7
```

```
The youngest child is SUnil
```